

Applications of PINNs in the search for finite-time singularities.

Maximilian Pochapski

December 1, 2025

Abstract

I investigated the use of physics-informed neural networks (PINNs) to search for finite-time singularities in PDEs related to the Navier-Stokes equations. In this report, I focusing on the one-dimensional Córdoba–Córdoba–Fontelos (CCF) model. Following the techniques introduced in Wang *et al.* (2023) [3], I implemented a self-similar ansatz parameterized by a blow-up exponent λ and trained neural networks to infer smooth initial profiles to finite-time singularities. The approach incorporates custom coordinate transforms, numerical Hilbert transforms, and multi-component loss functions tailored to the CCF equation. My numerical experiments recovered a candidate for $\lambda \approx 0.95$, albeit with low confidence. More resources would be required to find a better solution.

1 Introduction

Finite-time blow-up in the 3D Navier–Stokes equations is one of the seven Millenium Prize problems. If singularities can emerge from smooth initial conditions, it would signal a fundamental breakdown in the predictive power of these equations.

A leading candidate for such a finite-time singularity is the Hou–Lou scenario: an axisymmetric model of fluid motion inside a cylinder governed by the 3D Euler equations with smooth initial data. In their original 2014 paper [2], Hou and Lou reported numerical evidence suggesting the formation of a singularity, consistent with a self-similar blow-up developing in finite time.

More recently, a team at DeepMind used physics-informed neural networks (PINNs) as a universal approximator to infer the initial profiles in the Hou–Lou setup [6]. They employed a heuristic self-similar ansatz (similar to Eqn. 6) that incorporates a neural network to learn the initial field configuration. The ansatz is parameterized by a constant λ , and their aim is to identify admissible values of λ that can lead to unstable, singularity-forming solutions (Figure 1).

Their analysis used high precision, bespoke full-matrix gauss-newton optimizers, specialized update steps for λ , and large neural networks to discover values of λ up to $O(10^{-13})$ for the Cordoba-Cordoba-Fontelos (CCF) equation and other equations

related to Navier-Stokes. However, [6] is actually a high-precision refinement of the singularity-finding technique established in an earlier publication by the same author back in 2023 [3].

Therefore, in this analysis I will be following the work introduced in [3], which first introduced this method for discovering finite-time singularities.

2 Equations

The authors in [3] discovered values of λ related to stable and unstable finite-time singularities of the Boussinesq (2D), IPM (2D), and CCF (1D) equations. In this analysis I will be focusing on the CCF equations as a demonstration of the technique.

2.1 Cordoba-Cordoba-Fontelos (CCF)

The CCF equations model a Surface Quasi-Geostrophic (SQG) equation, a 2D system used in atmospheric science which is known to develop singularities in finite time [1][3]. The CCF equation is actually derived from the more general De-Gregorio equations (1) which are parametrized by a . For values $a < 0$, advection and vortex stretching work together to create finite-time singularities. When $a = -1$, you get the CCF equation (2):

$$\omega_t + au\omega_x = \omega u_x \quad (1)$$

$$\omega_t - u\omega_x = \omega u_x \quad (2)$$

Our goal is to encode a finite-time singularity into our ansatz; isolating the neural network to find a smooth initial profile responsible for the singularity. Using the self-similar Ansatz from [3], the solutions for the vorticity, $\omega(x, t)$, can be parameterized by λ :

$$\omega(x, t) = \frac{1}{1-t} \Omega \left(\frac{x}{(1-t)^{1+\lambda}} \right) \quad (3)$$

Then if we define the velocity $u = \int_0^y H[\Omega] ds$, where H is the Hilbert transform, and the change of coordinates $y = \frac{x}{(1-t)^{1+\lambda}}$, the full CCF equation becomes:

$$\Omega + ((1+\lambda)y - u) \frac{\partial \Omega}{\partial y} - \Omega \frac{\partial u}{\partial y} = 0 \quad (4)$$

The Hilbert transform requires a large domain to sample over; but, we need small physical numbers to stabilize the numerical simulation. To get around this, we perform the coordinate transform:

$$y = \sinh(z) \iff z = \sinh^{-1}(y), \quad y \in (-\infty, \infty) \rightarrow z \in [-30, 30]$$

Where we have chosen $\sinh(30) \approx 10^{12} \approx \infty$ for our boundary. (4) becomes:

$$\Omega(z) + ((1 + \lambda) \sinh(z) + aU(z)) \cosh^{-1}(z) \partial_z [\Omega(z)] - \Omega(z) \cosh^{-1}(z) \partial_z [U(z)] = 0 \quad (5)$$

For each field, in this case Ω and u , we construct independent, scalar Neural Networks that are incorporated into a function $q(z) : \mathbb{R} \rightarrow \mathbb{R}$:

$$q(z) = \left(\frac{NN_q(z) - NN_q(-z)}{2} \right) \cdot (1 + z^2)^{-1/(2(1+\lambda))} \quad (6)$$

We implicitly condition $q(z)$ to produce odd solutions and to decay at infinity to match the Hou-Lou scenario.

2.1.1 Loss Functions

The loss functions are split across the equation loss (the residuals of the solution away from the boundary), boundary loss (residuals of the solution at the boundary), and smoothness loss (higher order residuals near the origin to solve for a smooth initial profile). The full loss equation is written as:

$$J(y) = \hat{c}_s \left(\frac{1}{n_b} \sum_{j \in \{\Omega, u\}}^{n_b} \text{loss}_g^{(j)} \right) + \hat{c}_e \left(\frac{1}{n_e} \sum_{j \in \{\Omega, u\}}^{n_e} \text{loss}_f^{(j)} \right) + \hat{c}_s \left(\frac{1}{n_e} \sum_{j \in \{\Omega, u\}}^{n_e} \text{loss}_s^{(j)} \right) \quad (7)$$

Where j indexes over the fields $\{u, \Omega\}$. The boundary loss, $\text{loss}_g^{(j)}$, is broken down into:

$$g_1 = \Omega(0.5) + 0.05 \quad (8)$$

$$g_2 = \frac{1}{|Z_b|} \sum_{z \in Z_b} \Omega(z)^2 \quad (9)$$

$$\text{loss}_g = \frac{(g_1 + g_2)}{2} \quad (10)$$

g_1 lifts $q(z)$ away from the trivial solution, while g_2 ensures $q(z)$ decays at points near the boundary. In my analysis I used $|Z_b| = 20$ samples.

The field loss, $\text{loss}_f^{(j)}$ is decomposed into:

$$f_1 = \frac{1}{N_{|Z|}} \sum_{z \in Z_f} \Omega(z) + ((1 + \lambda) \sinh(z) + au(z)) \cosh^{-1}(z) \partial_z [\Omega(z)] - \Omega(z) \cosh^{-1}(z) \partial_z [u(z)] \quad (11)$$

$$f_2 = \frac{1}{N_{|Z|}} \sum_{z \in Z_f} \cosh^{-1}(z) \partial_z [u(z)] - H_n[\Omega(z)] \quad (12)$$

$$\text{loss}_f = \frac{(f_1 + f_2)}{2} \quad (13)$$

Where f_2 was derived from the definition $u_y = H\Omega$, and H_n is the numerical Hilbert transform. In my analysis, $Z_f \in [-30, 30]$ and $|Z_f| = 10,000$.

Lastly, the smoothness loss is given by:

$$\text{loss}_s = \frac{1}{N_s} \sum_{i=1}^{N_s} \left| \frac{d}{dz} (z_i, q(z_i)) \right|^2 \quad (14)$$

Where $N_s = 80$ in my analysis.

Across all losses, all samples were randomly generated within their respective ranges at every iteration.

2.2 Neural Network Hyperparameters

I used a simple, fully-connected neural network with 3 hidden layers and 20 units per hidden layer. I used $\tanh(z)$ as the activation function, and the final layer is an exponential activation function, similar to [3]. All functions were written in Python using the JAX library.

3 Analysis

3.1 Training

My code [7] mostly follows the process from [3], however, there are some crucial differences.

First, the similarities. My trainable parameters consisted of the neural network weights for both Ω and U , along with λ . I also used a two-stage training algorithm; a first pass with the ADAM optimizer followed by a refinement pass with L-BFGS.

Now, the differences. The authors implemented a custom, high-accuracy Hilbert numerical transform by modifying the first-order implementation found in [4]. Since their implementation was never listed, I made my own implementation using the Simpsons 3/8 rule. They also never mentioned the ranges for their sampling points, or the weights they used for the values of $\hat{c}_i$ in the loss function.

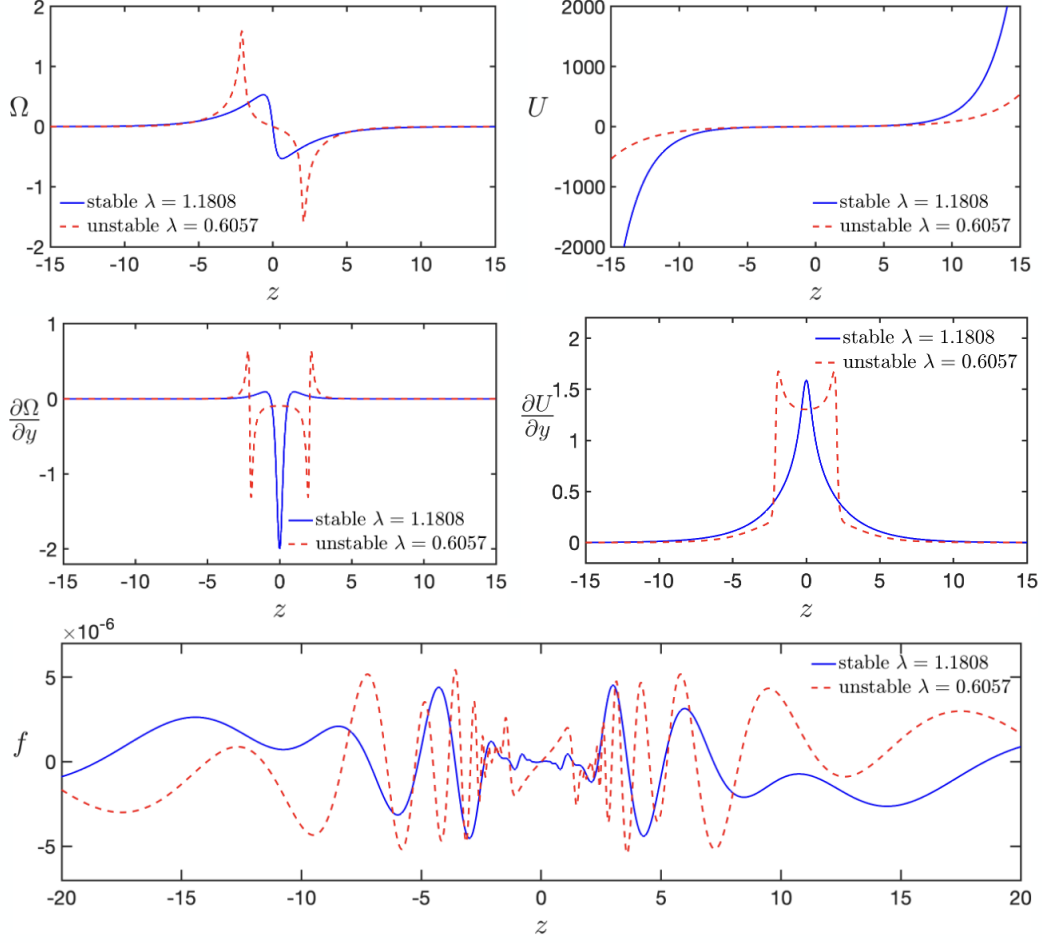


Figure 1: Plots of U, Ω , their respective derivatives, and the residuals of the field errors, loss_f , reprinted from [3]. For each plot two different values of λ are plotted. A solid blue plot is the result of a stable singularity $\lambda = 1.1808$, while a dotted red plot is the result of an unstable singularity $\lambda = 0.6057$. An unstable profile can be qualitatively characterized as having a sharper initial profile than a stable one.

In fact, most hyperparameters for their training runs were excluded, which makes reproducing their results extremely costly, and impossible for me given the limited scope of this project. Nevertheless, I will continue this report by presenting an analysis on the best training run I had while exploring the hyperparameter space; using 10,000 iterations of the ADAM optimizer and 40,000 iterations of L-BFGS, which took ~ 7 hours for a single run using a single-core CPU process. My best candidate result was $\lambda \approx 0.95$ Figure 2.

3.2 Stability Analysis

To determine if the solution for λ is stable or unstable, I conducted a stability analysis. Following [3], we introduce the perturbation terms:

$$\Omega = \hat{\Omega} + \Omega_\epsilon, U = \hat{U} + U_\epsilon \quad (15)$$

And define the linearization of the CCF equation around the approximate solution as:

$$\begin{aligned} \mathcal{L}\Omega_\epsilon = & -\Omega_\epsilon(z) - [(1 + \hat{\lambda}) \sinh(z) - \hat{U}(z)] \cosh^{-1}(z) \partial_z [\Omega_\epsilon(z)] \\ & + U_\epsilon \cosh^{-1}(z) \partial_z \hat{\Omega} + \hat{\Omega} H \Omega_\epsilon + \Omega_\epsilon \cosh^{-1}(z) \partial_z \hat{U} \end{aligned} \quad (16)$$

Where $U_\epsilon = \int_0^y H \Omega_\epsilon$. We look for the eigenvalues and eigenfunctions of the equation:

$$\mathcal{L}(\Omega_\epsilon, U_\epsilon) = \mu \Omega_\epsilon \quad (17)$$

Where $\mu \in \mathbb{C}$ is the eigenvalue of the solution. For each smooth solution to the CCF, there exist two trivial eigenvalues/eigenfunction corresponding to scaling, which are:

1. **Spatial Rescaling** – $\Omega_{\epsilon 0} = \tanh(z) \partial_z \hat{\Omega}$, where $\mu = 0$
2. **Temporal Translation** – $\Omega_{\epsilon 1} = \hat{\Omega} + (1 + \hat{\lambda}) \tanh(z) \partial_z \hat{\Omega}$, where $\mu = 1$

The results of the analysis are shown in Figure 3. For $\lambda \approx 0.951$ I was only able to achieve training residues on the order of 10^{-2} , which manifests in large errors between the linearized equations and the eigenfunctions. Since the residues of the trivial eigenpairs are so large, I can safely disregard λ as a potential candidate, and have to find another set of hyperparameters to run the training again. However, if the residuals were smaller, I would have continued the analysis like in [3] and searched for values of $\mu > 0$ which would have implied that the singularity formed by λ is unstable. Similarly, if I could find $\mu < 0$ then this would hint that the singularity might be stable.

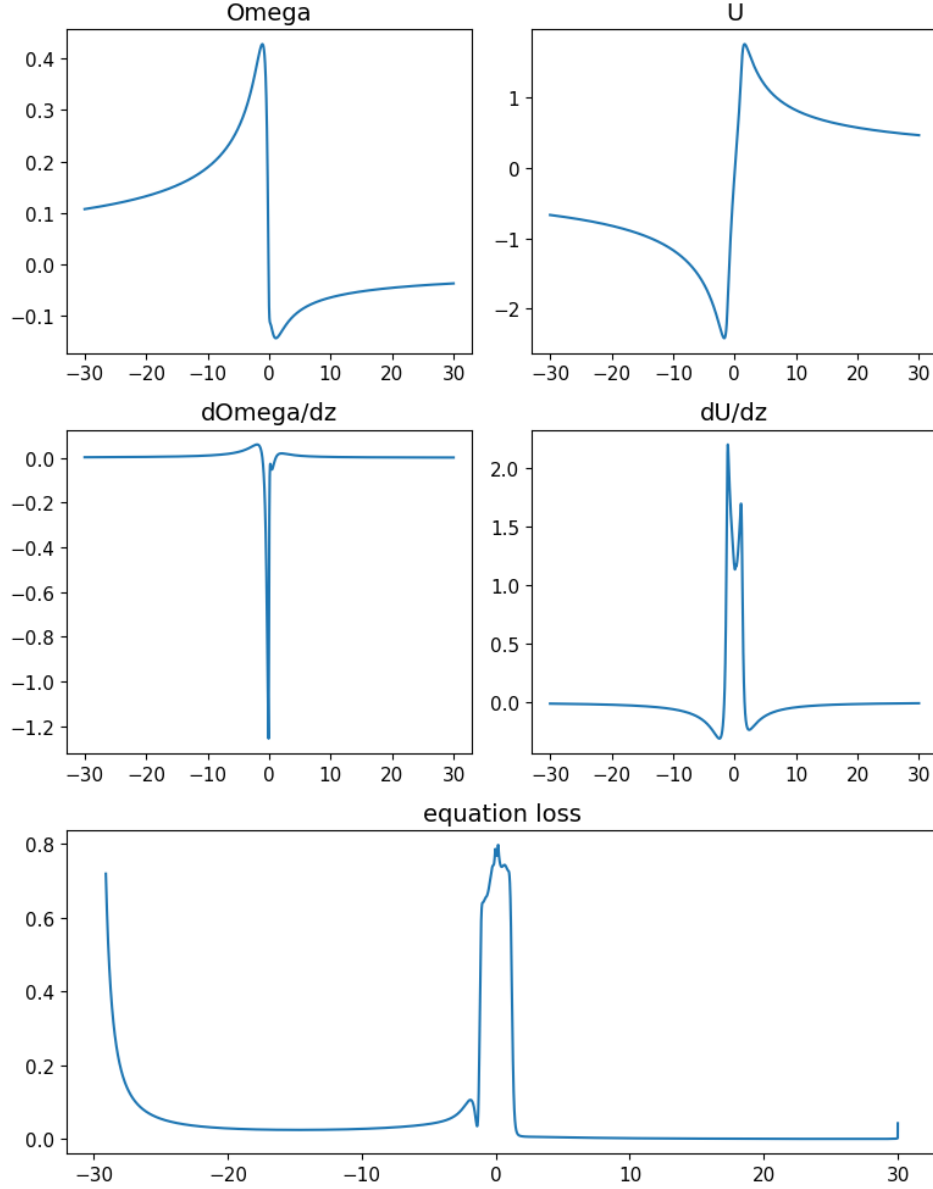


Figure 2: Best solution I was able to find thus far: $\lambda \approx 0.95$ using 10,000 iterations of ADAM followed by 40,000 iterations of L-BFGS. The figure for the equation loss was trimmed by 150 samples from the left to better illustrate the loss landscape. Compared to the results of Figure 1, my solutions are only faintly qualitatively similar to the unstable solutions for $\lambda = 0.6057$.

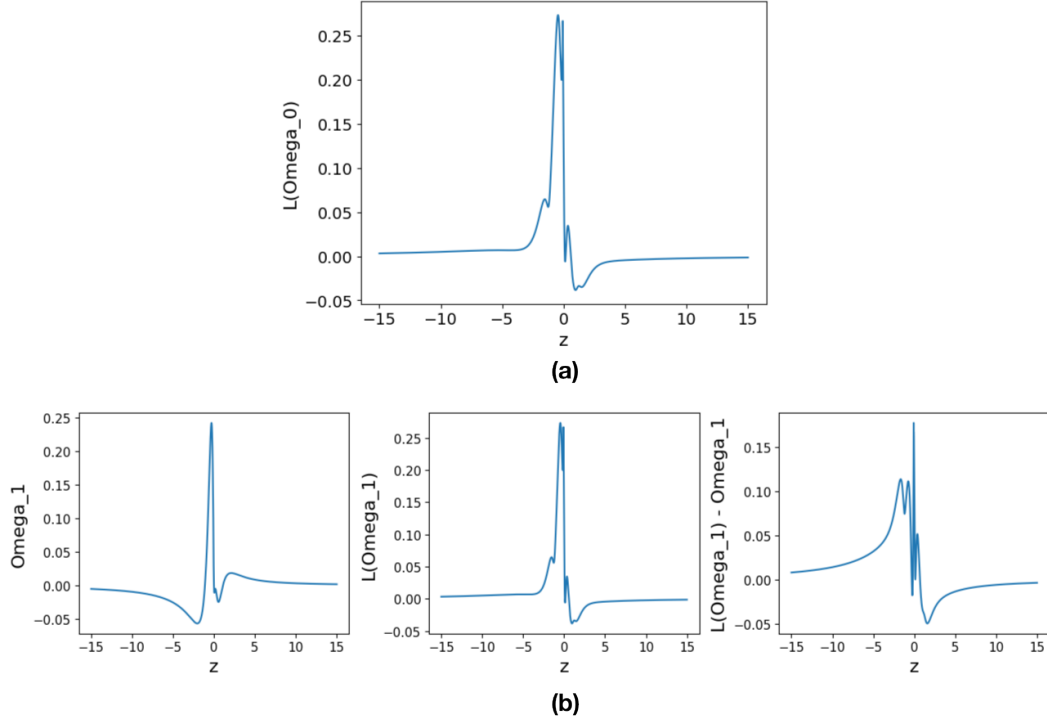


Figure 3: Plots of $\mathcal{L}\Omega_{\epsilon 0}$ and $\mathcal{L}\Omega_{\epsilon 1} - \Omega_{\epsilon 1}$ for candidate solution $\lambda \approx 0.95$. (a): A plot of $\mathcal{L}\Omega_{\epsilon 0} - \mu\Omega_{\epsilon 0}$, where $\mu = 0$. The large residuals of the PINN solution leads to a large error in the eigenfunction. (b) A similar story for $\Omega_{\epsilon 1}$. Plotted as a function of z , from left to right: The eigenfunction $\Omega_{\epsilon 1}$, the result of the linearized equation $\mathcal{L}(\Omega_{\epsilon}, U_{\epsilon})$ (Eqn. 16), and the residual $\mathcal{L}(\Omega_{\epsilon}, U_{\epsilon}) - \mu\Omega_{\epsilon 1}$, where $\mu = 1$.

4 Discussion and Conclusion

In the end, my largest (and best) run had 10,000 iterations using the ADAM optimizer followed by 40,000 iterations of the L-BFGS algorithm (Figure 2). For context, the authors of [3] used 100,000 and 250,000 iterations, respectively. Their training time took 16 hours on an A100 GPU. All of my simulations were completed on a single-core CPU, with most training runs averaging around ~ 7 hours each. This was a experimental trade-off I made given the time constraints; I figured I would spend the remaining time finding promising hyperparameters for λ directly, rather than spending the remaining time optimizing my code. Unfortunately, given the long run-times, I wasn't able to explore as many hyperparameters as I would have liked, or given enough time for my training runs to arrive at a better residual.

Given more time and compute resources, I believe I would have managed to match their solutions. This was my first project using JAX, which is a performant machine-learning library for Python as advertised in [6]. The code used to run this experiment can be found here [7].

In [3], the authors included λ in the trainable parameters during training, as did I, because it is the simplest strategy. However, most recently in [6] they have reflected that so-called *gradient training* is not effective for finding values of λ for unstable blow-up solutions to high numerical accuracy. More specialized methods, such as *λ -inference* and *funnel inference* can be found in the Supplementary of [6].

In conclusion, PINNs have proven to be a new powerful and flexible tool mathematical physicists can use to attack the Millenium Prize problem. I believe further advancements in computer-assisted proofs and deep learning research will continue to aid scientists in solving the worlds toughest challenges.

References

- [1] Córdoba, A., Córdoba, D., and Fontelos, M. A. Formation of singularities for a transport equation with nonlocal velocity. *Ann. of Math.* **162**, 1377-1389, 2005.
- [2] Luo, G. and Hou, T. Y. Potentially singular solutions of the 3D axisymmetric Euler equations. *Proc. Natl. Acad. Sci. U.S.A.* **111**, 12968-12973, 2014.
- [3] Wang, Y., Lai, C.-Y., Gómez-Serrano, J., and Buckmaster, T. Asymptotic Self-Similar Blow-Up Profile for Three-Dimensional Axisymmetric Euler Equations Using Neural Networks. *Phys. Rev. Lett.* **130**, 244002, 2023.
- [4] Bilato, R., Maj, O., and Brambilla, M. An algorithm for fast Hilbert transform of real functions. *Adv. Comput. Math.* **40**, 1159-1168, 2014.
- [5] Kiselev, A. Regularity and Blow up for Active Scalars. *Math. Model. Nat. Phenom.* **5**, 225-255, 2010.
- [6] Wang, Y., Bennani, M., Martens, J., et al. Discovery of Unstable Singularities. *arXiv preprint arXiv:2509.14185*, 2025.
- [7] Pochapski, M. Unstable-Singularities. *GitHub repository*, 2025. <https://github.com/mxpoch/Unstable-Singularities>